

Débugger son application

Table des matières

I. Contexte	3
II. Généralités sur le debugging	3
III. Exercice : Appliquez la notion	8
IV. Le debugger et les outils pour React Native	8
V. Exercice : Appliquez la notion	12
VI. Essentiel	15
VII. Auto-évaluation	15
A. Exercice final	15
B. Exercice : Défi	16
Solutions des exercices	18

I. Contexte

Durée : 1 h

Environnement de travail : Une application React Native initialisée sur son poste

Pré-requis : Environnement de développement Expo en local

Contexte

Lorsque l'on développe une application, il peut être parfois intéressant de se positionner à un endroit dans le code et de notifier la valeur des variables, ou de carrément bloquer l'exécution et déclencher l'avancement des instructions pas à pas.

Ce procédé s'appelle le **debugging** et permet donc de tracer d'éventuels comportements indésirables dans notre application ou simplement de vérifier le bon fonctionnement de nos algorithmes.

Pour utiliser le debugger dans React Native, c'est très simple, et nous allons voir comment procéder dans ce chapitre dédié à son utilisation.

II. Généralités sur le debugging

Objectifs

- Comprendre les différents outils mis à notre disposition pour tracer du code
- Comprendre la logique de traçage de code

Mise en situation

Tout au long du développement d'une application, il est nécessaire de s'assurer du bon fonctionnement du code réalisé. Souvent, la réalisation de tests fonctionnels peut faire apparaître des comportements non souhaités, des bugs. Mais quelques fois, elle n'est pas suffisante pour s'assurer que tout fonctionne comme voulu. Nous allons voir qu'il existe des outils permettant d'analyser en temps réel le comportement de l'application afin de trouver la source des dysfonctionnements. Nous allons aborder les principes généraux du debugging avant d'en illustrer le fonctionnement.

Prévenir les bugs

Le nombre de bugs dans une application est directement lié aux pratiques de développement mises en place à l'initialisation du projet. En effet, le respect de bonnes pratiques peut permettre d'éviter certains bugs, mais aussi d'en comprendre plus rapidement l'origine, et facilite la mise en place d'une solution corrective. Parmi ces recommandations, on retrouve la nécessité :

- **D'écrire des spécifications claires** : le comportement d'une fonctionnalité peut être considéré différemment d'une personne à l'autre selon les informations en leur possession sur la fonctionnalité attendue. Par conséquent, l'écriture de spécifications claires et structurées permet de réduire les quiproquos et assure une bonne compréhension entre les différents participants à la réalisation du projet.
- **De respecter les principes de base pour coder proprement** : cela passe par le nommage de variables claires et compréhensibles, l'écriture de fonctions n'ayant qu'une seule responsabilité, puis par des notions d'organisation de code, comme nous l'avons vu dans un autre chapitre.

Malgré cela, la nécessité de suivre le comportement d'une fonctionnalité en direct afin d'en analyser le comportement reste présente.

Logique de debugging

Derrière la notion de debugging se trouve l'idée de trouver l'origine d'un comportement inattendu afin de le corriger. Pour cela, la méthodologie la plus courante est une approche itérative de l'analyse du code, c'est-à-dire la répétition de vérifications des comportements des éléments du code jusqu'à trouver celui responsable du bug.

Afin de mettre en œuvre cette approche, une solution est de se positionner avant et après un bloc d'instructions que l'on pense défectueux, afin d'en faire l'analyse, puis de répéter l'opération en réduisant les bornes de départ et de fin jusqu'à trouver l'origine du dysfonctionnement.

L'utilisation de la console

Pour analyser le fonctionnement de différentes instructions, il est utile d'afficher des messages servant de repères ou permettant de connaître la valeur des éléments les constituant. Pour cela, on peut utiliser la console JavaScript qui permet d'afficher différentes informations dans le terminal dans lequel l'application est exécutée.

L'affichage de messages se fait par l'utilisation de la fonction `console.log()`, qui accepte autant de paramètres que l'on veut et qui en fait la concaténation en chaîne de caractères.

Mise en pratique

Afin d'illustrer les étapes nécessaires à la découverte de l'origine d'un bug, nous allons utiliser l'exemple de code suivant et présenter une démarche permettant de corriger un dysfonctionnement.

Exemple

```

1 import React, { useState } from "react";
2 import { View, Text, StyleSheet } from "react-native";
3 import Constants from "expo-constants";
4 import { TextInput } from "react-native-gesture-handler";
5
6 const NUMBER_TO_FOUND = 6;
7
8 export default function App() {
9   const [counter, setCounter] = useState();
10
11   return (
12     <View style={styles.container}>
13       <TextInput
14         placeholder="Saisir le nombre pour essayer de trouver le nombre mystère"
15         value={counter}
16         onChangeText={(val) => setCounter(Number(val))}
17       />
18       <Text style={styles.text}>
19         {getMessageFromNumber(NUMBER_TO_FOUND, counter)}
20       </Text>
21     </View>
22   );
23 }
24
25 function getMessageFromNumber(numberToFound, currentValue) {
26   if (numberToFound === currentValue) {
27     return "Bravo vous avez trouvé le nombre mystère !";
28   }
29
30   return numberToFound < currentValue ? "C'est moins !" : "C'est plus";
31 }
32

```

```

33 const styles = StyleSheet.create({
34   container: {
35     flex: 1,
36     paddingTop: Constants.statusBarHeight,
37   },
38   text: {
39     textAlign: "center",
40     fontSize: 24,
41     marginVertical: 20,
42   },
43 });

```

[cf. debug1]

À travers cet exemple, on constate que, lorsque le champ de saisie est vide, l'affichage indique « *C'est plus* », ce qui n'est pas le comportement attendu. Afin de proposer une correction, il est nécessaire d'identifier l'origine de ce fonctionnement en analysant le bloc d'instructions se chargeant de l'affichage du message, c'est-à-dire la fonction `getMessageFromNumber(numberToFound, currentValue)`.

Afin de faire l'analyse du fonctionnement de cette fonction, nous allons pouvoir afficher des messages dans la console :

```

1 function getMessageFromNumber(numberToFound, currentValue) {
2   console.log("Number to found = ", numberToFound);
3   console.log("Current value = ", currentValue);
4
5   console.log(
6     "numberToFound === currentValue : ",
7     numberToFound === currentValue
8   );
9
10  if (numberToFound === currentValue) {
11    return "Bravo vous avez trouvé le nombre mystère !";
12  }
13
14  console.log("numberToFound > currentValue : ", numberToFound > currentValue);
15
16  return numberToFound < currentValue ? "C'est moins !" : "C'est plus";
17 }

```

On constate que, lors du premier affichage de l'application, les messages suivants s'affichent dans la console :

```

1 Number to found = 6
2 Current value = undefined
3 numberToFound === currentValue : false
4 numberToFound > currentValue : false

```

Il apparaît ainsi que la variable `currentValue` vaut `undefined` lors de l'initialisation de l'application. On peut à présent proposer une correction qui renvoie une chaîne de caractères vide si cette variable n'a pas de valeur :

```

1 [...]
2 function getMessageFromNumber(numberToFound, currentValue) {
3   if(typeof currentValue === "undefined" || currentValue === null){
4     return ""
5   }
6
7   if (numberToFound === currentValue) {
8     return "Bravo vous avez trouvé le nombre mystère !";
9   }
10

```

```
11 return numberToFound < currentValue ? "C'est moins !" : "C'est plus";
12 }
13 [...]
```

L'objet `console` de JavaScript expose également d'autres fonctions, comme `console.warn()` et `console.error()` qui permettent respectivement l'affichage de messages d'avertissement ou d'erreur directement sur la page de l'application.

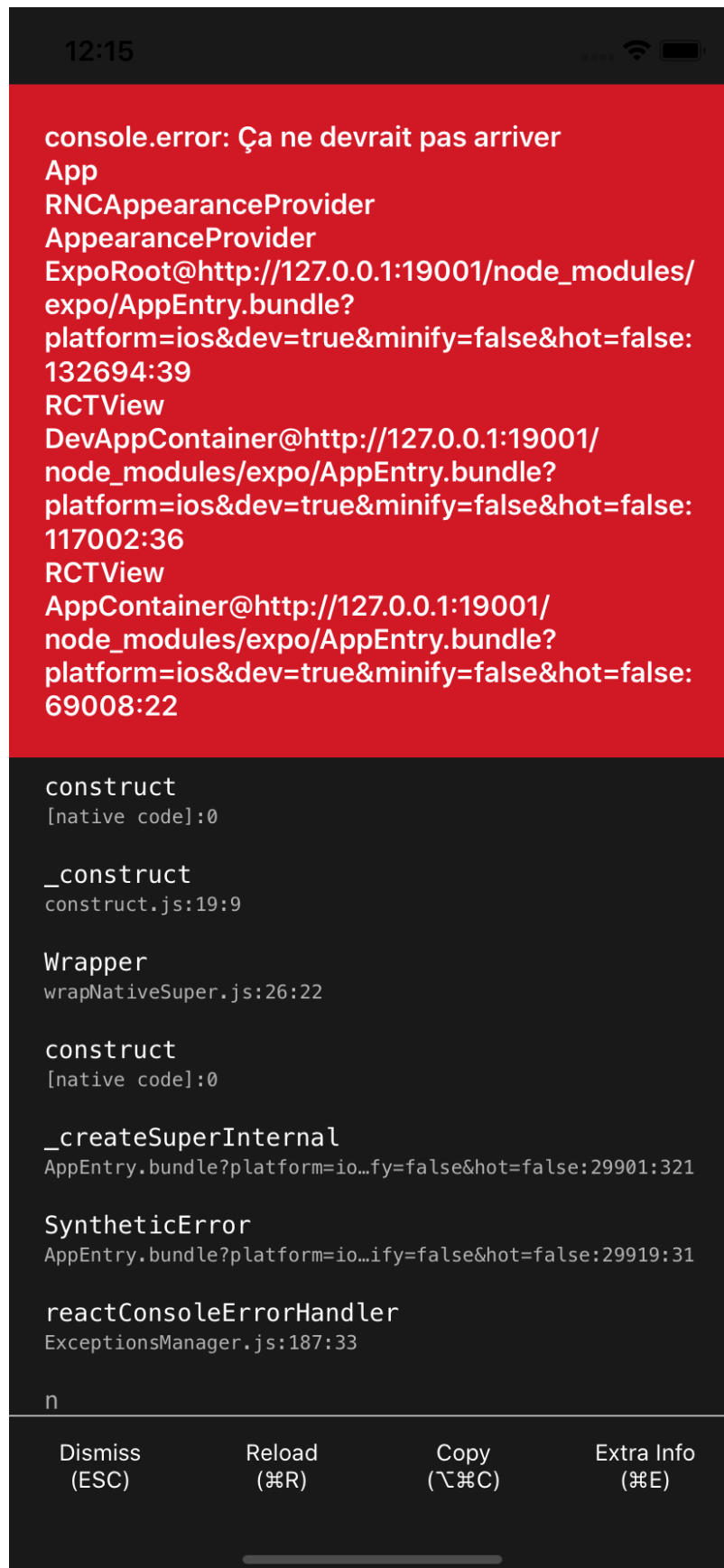
12:15



Saisir le nombre pour essayer de trouver le nombre mystère

Dismiss All

Ça ne devrait pas arriver



Syntaxe **À retenir**

- Afin de limiter le nombre de bugs présents dans une application, il est vivement recommandé d'appliquer les bonnes pratiques de conception de logiciels et de rédiger des spécifications claires et précises.
- Lorsque l'application présente des défaillances, on peut traquer les comportements défectueux en utilisant des outils comme `console.log()` afin d'afficher des messages : `console.warn()` pour afficher un avertissement ou `console.error()` pour afficher une erreur.

III. Exercice : Appliquez la notion

Question

Améliorez ce morceau de code pour prévenir le développeur dans deux cas :

- S'il n'y a pas de valeur dans les champs, il faut afficher un warning et renvoyer `null`
- Si la division se fait par 0, il faut afficher une erreur et renvoyer `null`

```

1 import React, { useState } from "react";
2 import { View, Text, TextInput } from "react-native";
3
4 export default function App() {
5   const [valueA, setValueA] = useState();
6   const [valueB, setValueB] = useState();
7
8   return (
9     <View style={{ padding: 50 }}>
10      <TextInput
11        placeholder="Valeur A"
12        value={valueA}
13        onChangeText={(val) => setValueA(val)}
14      />
15      <TextInput
16        placeholder="Valeur B"
17        value={valueB}
18        onChangeText={(val) => setValueB(val)}
19      />
20      <Text>{divide(valueA, valueB)}</Text>
21    </View>
22  );
23 }
24
25 function divide(a, b) {
26   return Number(a) / Number(b);
27 }

```

IV. Le debugger et les outils pour React Native

Objectif

- S'initier au debugging avec React Native et JavaScript

Mise en situation

Afin de faciliter les opérations de debugging, il est possible d'utiliser un outil très pratique appelé « debugger ». Celui-ci permettra d'exécuter du code pas à pas et de voir la valeur des variables en direct.

React Native propose d'exécuter le code JavaScript de l'application dans le moteur du debugger (dans notre cas, le debugger de Chrome) plutôt que dans le moteur de l'application. De cette manière, on va pouvoir interagir directement pendant le fonctionnement de l'application.

Activer le debugger

Pour activer le debugger, il faut afficher le menu de développement en secouant son téléphone ou en affichant le panneau de développement d'Expo. Selon l'environnement de travail et le terminal cible, la manière peut varier :

- Pour un terminal iOS : il faut secouer le téléphone ou toucher l'écran avec trois doigts.
- Pour le simulateur iOS : la combinaison **Ctrl-Cmd-Z** ou bien la combinaison **Cmd+D**.
- Pour un terminal Android : il faut secouer le terminal verticalement ou lancer `adb shell input keyevent 82` dans un terminal si le téléphone est connecté via USB.
- Sur l'émulateur Android : la combinaison **Cmd+M** ou la commande `adb shell input keyevent 82`.

Remarque

L'interface du debugger peut varier d'un navigateur à l'autre. Cependant, les concepts sont généralement les mêmes et les boutons sont très semblables.

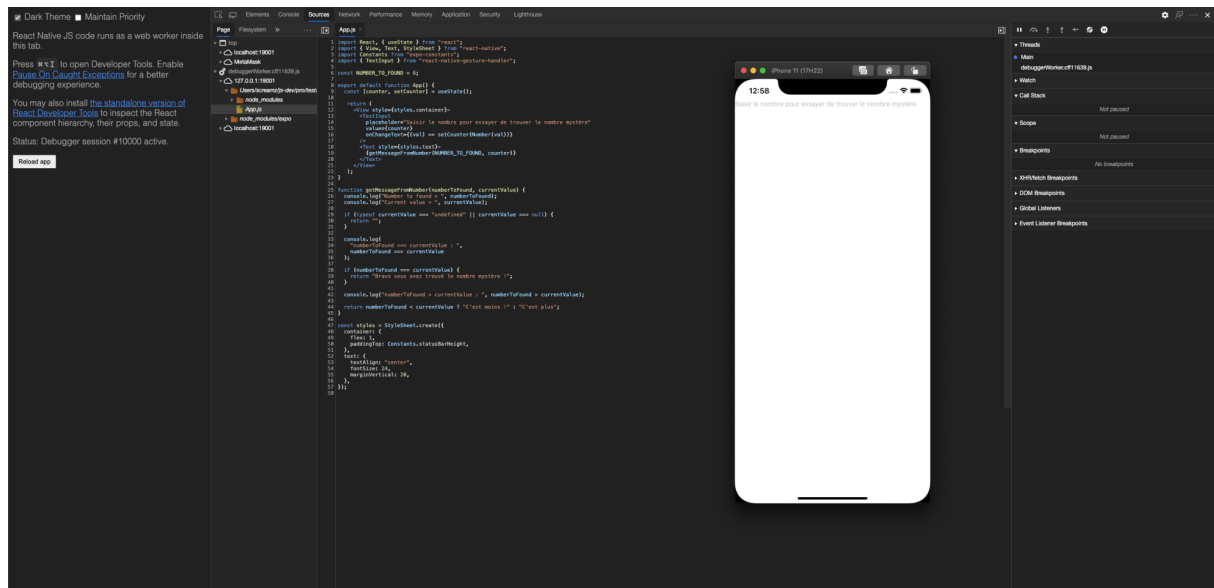
[cf. debug4]

Une fois le menu développeur ouvert, il faut choisir l'option *Debug Remote JS* qui ouvre un navigateur à l'URL `http://localhost:19001/debugger-ui/`. Pour accéder aux fonctionnalités du debugger, il faut maintenant ouvrir le panneau des outils de développement du navigateur, soit par un *clic droit* > *Inspecter* sur la page affichée, soit par le raccourci associé (f12 pour Firefox et Chrome, par exemple), puis aller sur l'onglet *Sources*.

On retrouve ici tous les fichiers de l'application et, lorsque l'on sélectionne un fichier, l'interface se décompose alors en trois parties :

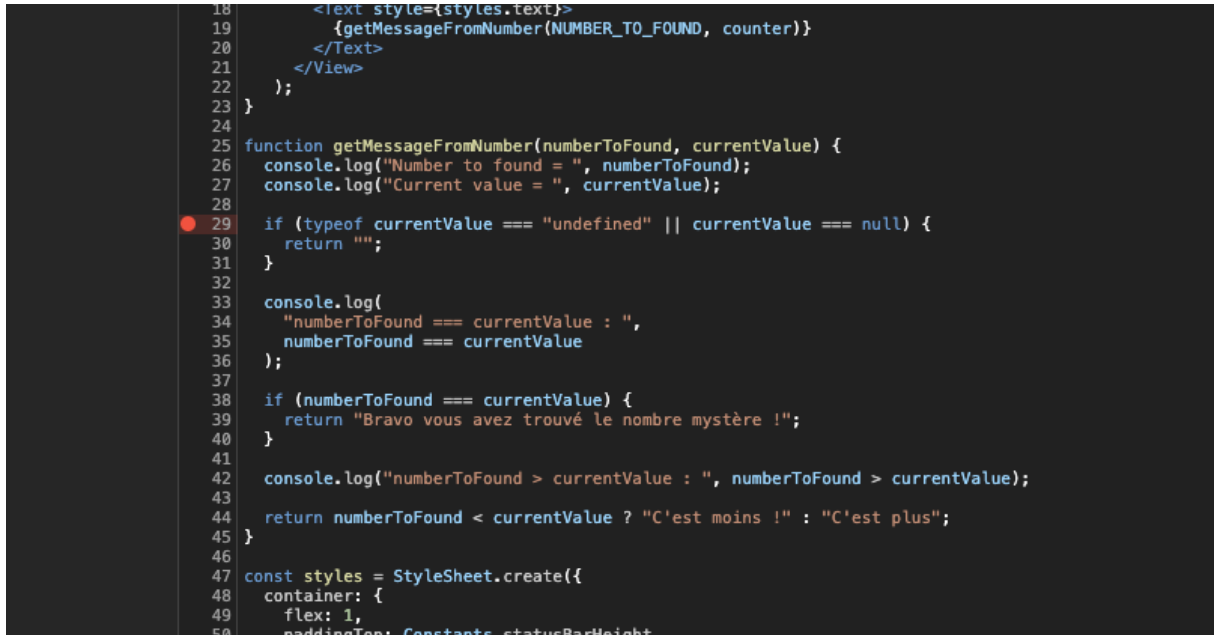
- À gauche, l'explorateur de fichiers
- Au centre, le code du fichier ouvert
- Et à droite, le panneau de contrôle

Par exemple, en sélectionnant le fichier `App.js`, on obtient :

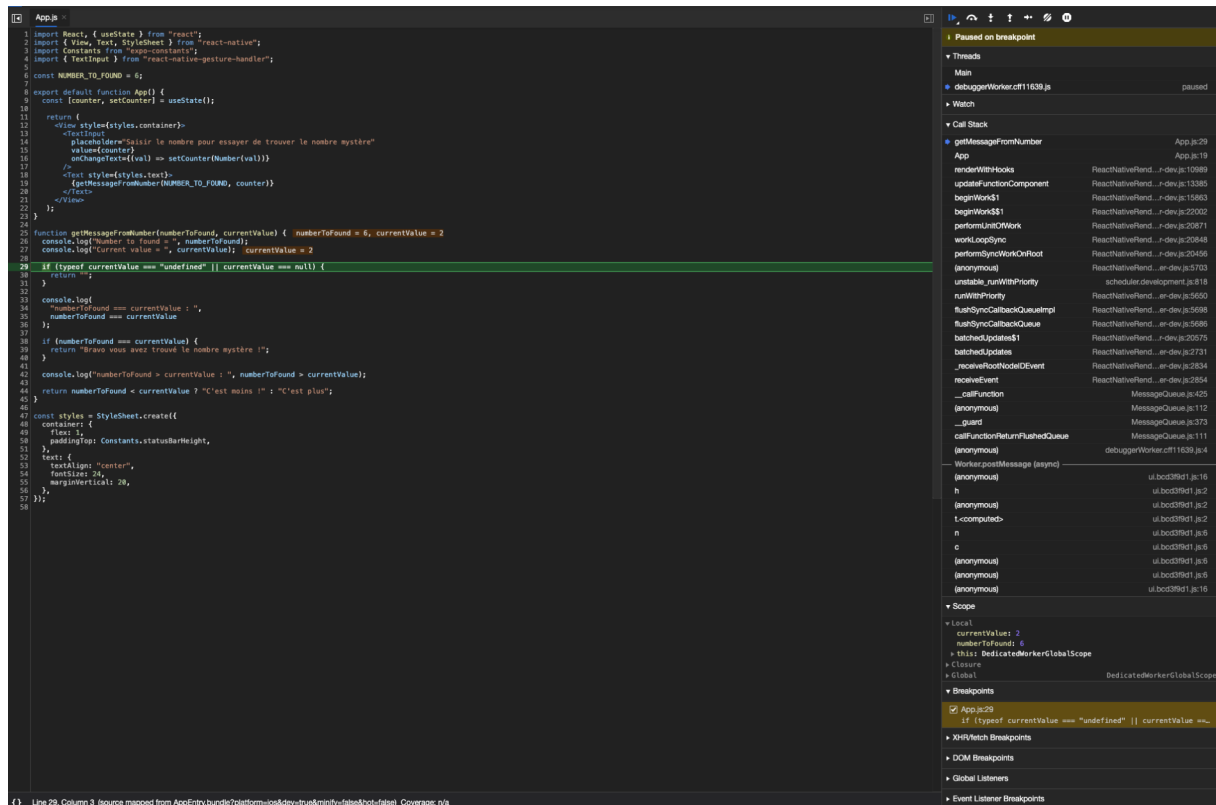


Utiliser des breakpoints

Une nouvelle approche du debugging est l'utilisation de points d'arrêt (*breakpoints*) dans le debugger. Un point d'arrêt indique au debugger qu'il doit interrompre l'exécution du code sur une ligne spécifique. Cela peut se faire directement par une instruction JavaScript, `debugger`, ou par sélection, au clic, d'une ligne dans l'interface du debugger.



Lorsque l'exécution du code arrive sur l'instruction marquée d'un point d'arrêt, celle-ci s'interrompt et il devient possible de consulter ou modifier en direct les valeurs des différentes variables, ou bien de suivre l'exécution du code pas à pas.



Le mode pas à pas

Lorsque l'exécution est interrompue par un point d'arrêt, le debugger propose différentes actions pour interagir avec l'application, notamment par l'utilisation de boutons permettant d'en contrôler l'exécution.

Ainsi, il est possible, grâce au bouton **Play**, de reprendre l'exécution normale de l'application, tandis que les suivants permettent une gestion fine du suivi de l'exécution.

Il est donc possible de suivre l'avancement de l'exécution des instructions ligne par ligne, de décider si l'on souhaite entrer dans une fonction ou en sortir directement pour retourner à l'instruction appelante.

Ces opérations sont gérées par les boutons suivants :

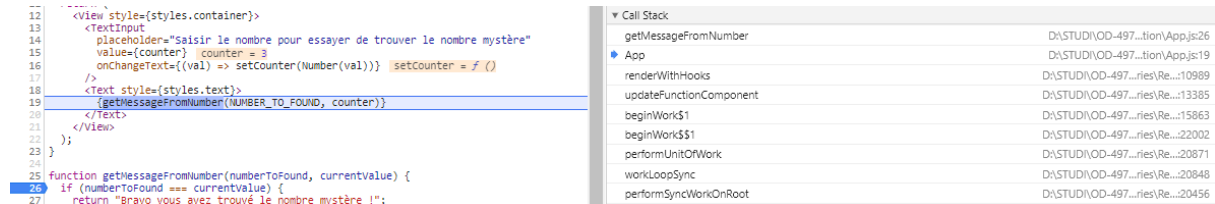


Sur l'image présentée, dans l'ordre, on trouve le bouton **Play**, un bouton permettant de sauter une fonction, un bouton pour entrer dans une fonction, un bouton pour sortir d'une fonction et un bouton permettant d'avancer ligne par ligne. Les boutons suivants permettent respectivement de désactiver tous les breakpoints existants et d'interrompre l'exécution si une Exception est générée.

La pile d'appels

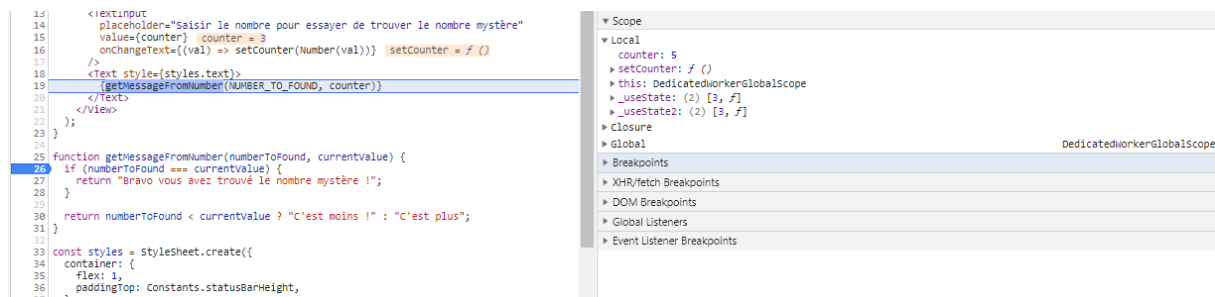
Le panneau à droite de l'interface du panneau des outils de développement du navigateur se découpe en plusieurs parties : parmi elles, on trouve la pile d'appels (*call stack*) correspondant à la ligne où l'exécution s'est interrompue.

Cette pile d'appels permet de visualiser rapidement le chemin suivi afin d'arriver à l'instruction en cours. Ainsi, lorsque de nombreuses fonctions se sont succédé, on peut retrouver rapidement l'origine de l'appel de cette dernière.



Le scope

Le panneau des outils de développement du navigateur contient aussi une partie répertoriant l'état local correspondant à la ligne d'instruction en cours, le *scope*. On y retrouve toutes les variables, avec leur valeur en cours par exemple. Il est aussi possible de modifier à la volée la valeur d'une variable afin de constater les variations de comportement lorsque l'on reprend l'exécution.



Syntaxe À retenir

- Le debugger est un outil très pratique pour suivre le déroulement de ses algorithmes dans l'application. Le code ne s'exécute plus sur le terminal, mais directement dans la machine virtuelle JavaScript qui est embarquée dans le navigateur web.
- Le debugger permet notamment de naviguer pas à pas et de récupérer la valeur des variables à l'endroit où l'on définit des points d'arrêt (*breakpoints*).

V. Exercice : Appliquez la notion

Question

Pour cet exercice, il vous est demandé de coder la fonction `buildLikeSentence`, dont le squelette vide est donné ci-dessous.

Il est interdit de modifier du code à l'extérieur de cette fonction, qui doit renvoyer une chaîne de caractères.

Vous utiliserez le debugger et/ou de l'affichage de messages dans la console pour vous assister dans la réalisation de cette fonction.

```
1 import React from "react";
2 import { View, Text } from "react-native";
```

```
3
4 export default function App() {
5   return (
6     <View style={{ padding: 50 }}>
7       <Text>{buildLikeSentence(true, 1)}</Text>
8       <Text>{buildLikeSentence(true, 2)}</Text>
9       <Text>{buildLikeSentence(true, 3)}</Text>
10      <Text>{buildLikeSentence(false, 0)}</Text>
11      <Text>{buildLikeSentence(false, 1)}</Text>
12      <Text>{buildLikeSentence(false, 2)}</Text>
13    </View>
14  );
15 }
16
17 function buildLikeSentence(userHasLiked, likersCount) {
18   // TODO: Il faut coder le corps de cette fonction en utilisant le debugger
19   return "À vous de jouer !";
20 }
```

L'objectif attendu de la fonction est de construire les textes suivants :

3:36

Vous avez aimé cet article
Vous et 1 autre avez aimé cet article
Vous et 2 autres avez aimé cet article
Personne n'aime cet article
1 autre a aimé cet article
2 autres ont aimé cet article



VI. Essentiel

Même si l'idéal est d'éviter d'avoir des bugs dans notre code, il est possible de le debugger : React Native met à disposition plusieurs outils pour cela.

Le plus simple est d'utiliser la console JavaScript, qui permet d'afficher un message lorsque l'interpréteur de code passe sur l'instruction. On peut également profiter de messages qui se répercutent sur l'interface en utilisant `console.warn` et `console.error`.

Lorsque l'on souhaite aller plus loin, on peut utiliser le navigateur web et son debugger JavaScript pour exécuter le code pas à pas et tracer l'exécution. De cette manière, on peut rentrer en détails dans l'algorithme et trouver les problèmes très simplement.

VII. Auto-évaluation

A. Exercice final

Exercice

Exercice

Pour traquer un bug, il est recommandé de...

- ☐ Procéder par encerclement, réduction et élimination
- ☐ Procéder par recherche aléatoire

Exercice

Pour afficher un message dans la console, j'utilise...

- ☐ `console.logMessage`
- ☐ `console.message`
- ☐ `console.log`

Exercice

Si j'utilise `console.warn`, que se passe-t-il ?

- ☐ J'obtiens un message rouge dans l'interface graphique de React Native uniquement
- ☐ J'obtiens un message orange avec la *stacktrace* dans la console et un panneau orange dans l'interface de l'application React Native en développement
- ☐ J'obtiens un message orange dans la console du packager uniquement

Exercice

Pour ouvrir le panneau de développement sur React Native iOS sur un vrai terminal, je peux...

- ☐ Appuyer avec trois doigts sur l'écran
- ☐ *Swiper* depuis la gauche au milieu jusqu'en haut à gauche
- ☐ Secouer le téléphone

Exercice

Où s'exécute le code JavaScript lorsqu'on l'utilise le mode **Remote debug** avec Google Chrome ?

- ☐ Il s'exécute dans l'interpréteur du terminal concerné
- ☐ Il s'exécute dans l'interpréteur V8 du navigateur web Chrome
- ☐ Il s'exécute dans le Cloud sur les serveur de Microsoft

Exercice

Comment place-t-on un `breakpoint` dans le code ?

- ☐ En cliquant dans le debugger à côté de la ligne sur laquelle on souhaite s'arrêter
- ☐ En ajoutant l'instruction `debugger` dans le code

Exercice

Via le debugger, je peux avancer ligne par ligne dans le code.

- ☐ Vrai
- ☐ Faux

Exercice

Qu'est-ce que la *call stack* ?

- ☐ C'est la liste qui présente l'ordre d'imbrication des appels de fonctions qui ont mené le programme au point où il en est
- ☐ C'est la liste des variables utilisées dans le scope courant

Exercice

On peut naviguer dans la *call stack*.

- ☐ Vrai
- ☐ Faux, elle n'est là qu'à titre informatif

B. Exercice : Défi

Maintenant que nous savons utiliser le debugger, utilisons nos connaissances acquises pour debugger le programme suivant qui présente plusieurs bugs.

Question

Il faudra pour cela utiliser le debugger et/ou l'affichage de messages dans la console.

Ce programme doit afficher un premier écran qui demande de saisir un nombre à trouver, puis un bouton **Lancer le jeu**. Ensuite, dans un deuxième écran, l'utilisateur doit saisir une valeur pour essayer de trouver le nombre et reçoit un indice qui lui dit si le nombre est plus grand, plus petit ou si c'est le bon nombre.

Quand il trouve le bon nombre, un bouton **Réinitialiser** permet de relancer la partie.

Vous trouverez ci joint le code actuel de l'application :

```
1 import React, { useState } from "react";
2 import { View, Text, StyleSheet, Button } from "react-native";
3 import Constants from "expo-constants";
4 import { TextInput } from "react-native-gesture-handler";
5
6 export default function App() {
7   const [numberToFoundInput, setNumberToFoundInput] = useState();
8   const [numberToFound, setNumberToFound] = useState(null);
9   const [userAnswerInput, setUserAnswerInput] = useState();
10 }
```



```

11  if (typeof numberToFound === "null") {
12      return (
13          <View style={styles.container}>
14              <TextInput
15                  key="choose-number-to-found"
16                  placeholder="Saisir le nombre mystère qu'il faudra trouver"
17                  value={numberToFoundInput}
18                  onChangeText={(val) => setNumberToFoundInput(val)}
19              />
20              <Button
21                  title="Lancer le jeu"
22                  onPress={() => {
23                      setNumberToFound(Number(numberToFoundInput));
24                  }}
25              />
26          </View>
27      );
28  }
29
30  const numberFoundByUser =
31      getMessageFromNumber(numberToFound, userAnswerInput) ===
32      "Bravo vous avez trouvé le nombre mystère";
33
34  return (
35      <View style={styles.container}>
36          <TextInput
37              key="user-answer"
38              placeholder="Saisir le nombre pour essayer de trouver le nombre mystère"
39              value={userAnswerInput}
40              onChangeText={(val) => setUserAnswerInput(val)}
41          />
42          <Text style={styles.text}>
43              {getMessageFromNumber(numberToFound, userAnswerInput)}
44          </Text>
45          {numberFoundByUser && (
46              <Button
47                  title="Réinitialiser"
48                  onPress={() => {
49                      setNumberToFound(null);
50                      setUserAnswerInput(null);
51                      setNumberToFoundInput(null);
52                  }}
53              />
54          )}
55      </View>
56  );
57 }
58
59 function getMessageFromNumber(numberToFound, currentValue) {
60     if (typeof currentValue === "undefined" || currentValue === null) {
61         return "";
62     }
63
64     if (numberToFound === currentValue) {
65         return "Bravo vous avez trouvé le nombre mystère !";
66     }
67

```

```
68   return numberToFound >= currentValue ? "C'est moins !" : "C'est plus";
69 }
70
71 const styles = StyleSheet.create({
72   container: {
73     flex: 1,
74     paddingTop: Constants.statusBarHeight,
75   },
76   text: {
77     textAlign: "center",
78     fontSize: 24,
79     marginVertical: 20,
80   },
81 });
```

Ci-dessous le résultat attendu :

[cf. debug10]

Solutions des exercices

Exercice p. Solution n°1

```

1 import React, { useState } from "react";
2 import { View, Text, TextInput } from "react-native";
3
4 export default function App() {
5   const [valueA, setValueA] = useState();
6   const [valueB, setValueB] = useState();
7
8   return (
9     <View style={{ padding: 50 }}>
10      <TextInput
11        placeholder="Valeur A"
12        value={valueA}
13        onChangeText={(val) => setValueA(val)}
14      />
15      <TextInput
16        placeholder="Valeur B"
17        value={valueB}
18        onChangeText={(val) => setValueB(val)}
19      />
20      <Text>{divide(valueA, valueB)}</Text>
21    </View>
22  );
23 }
24
25 function divide(a, b) {
26   if (isNil(a) || isNil(b)) {
27     console.warn("Il n'y a pas deux valeurs pour la division...");
28     return null;
29   }
30
31   if (Number(b) === 0) {
32     console.error("La division par zéro n'est pas possible ! Attention !");
33     return null;
34   }
35
36   return Number(a) / Number(b);
37 }
38
39 function isNil(value) {
40   return typeof value === "undefined" || (value === null) | (value === "");
41 }

```

Exercice p. Solution n°2

```

1 import React from "react";
2 import { View, Text } from "react-native";
3
4 export default function App() {
5   return (
6     <View style={{ padding: 50 }}>
7       <Text>{buildLikeSentence(true, 1)}</Text>
8       <Text>{buildLikeSentence(true, 2)}</Text>
9       <Text>{buildLikeSentence(true, 3)}</Text>

```

```

10     <Text>{buildLikeSentence(false, 0)}</Text>
11     <Text>{buildLikeSentence(false, 1)}</Text>
12     <Text>{buildLikeSentence(false, 2)}</Text>
13 </View>
14 );
15 }
16
17 function buildLikeSentence(userHasLiked, likersCount) {
18     const likerCountWithoutUser = userHasLiked ? likersCount - 1 : likersCount;
19
20     // Empty sentence if no likes
21     if (likersCount <= 0) {
22         return "Personne n'aime cet article";
23     }
24
25     switch (userHasLiked) {
26         case true:
27             return likerCount > 1
28                 ? `Vous et ${likerCountWithoutUser} autre${
29                     likerCountWithoutUser > 1 ? "s" : ""
30                 } avez aimé cet article`
31                 : "Vous avez aimé cet article";
32         case false:
33             return `${likerCountWithoutUser} autre${likersCount > 1 ? "s" : ""} ${
34                 likerCount > 1 ? "ont" : "a"
35             } aimé cet article`;
36     }
37 }

```

Exercice p. 15 Solution n°3

Exercice

Pour traquer un bug, il est recommandé de...

- ☒ Procéder par encerclement, réduction et élimination
- ☐ Procéder par recherche aléatoire
-  Pour suivre un bug, il est recommandé d'essayer de cibler la portion du code la plus petite possible qui serait susceptible de contenir le bug. Ensuite, il faut tracer l'exécution du code et voir à quel moment le bug intervient.

Exercice

Pour afficher un message dans la console, j'utilise...

- ☐ console.logMessage
- ☐ console.message
- ☒ console.log
-  On peut utiliser `console.log`, qui est la seule syntaxe existante.

Exercice

Si j'utilise `console.warn`, que se passe-t-il ?

- ☐ J'obtiens un message rouge dans l'interface graphique de React Native uniquement
- ☒ J'obtiens un message orange avec la *stacktrace* dans la console et un panneau orange dans l'interface de l'application React Native en développement
- ☐ J'obtiens un message orange dans la console du packager uniquement
-  `console.warn` affiche le message dans la console du packager, mais également dans l'interface visuelle de React Native, pour permettre au développeur de se rendre compte du warning très rapidement.

Exercice

Pour ouvrir le panneau de développement sur React Native iOS sur un vrai terminal, je peux...

- ☒ Appuyer avec trois doigts sur l'écran
- ☐ *Swiper* depuis la gauche au milieu jusqu'en haut à gauche
- ☒ Secouer le téléphone
-  Utiliser trois doigts ou secouer le téléphone sont les seules fonctions supportées pour ouvrir le panneau de développement sur un vrai terminal iOS.

Exercice

Où s'exécute le code JavaScript lorsqu'on l'on utilise le mode **Remote debug** avec Google Chrome ?

- ☐ Il s'exécute dans l'interpréteur du terminal concerné
- ☒ Il s'exécute dans l'interpréteur V8 du navigateur web Chrome
- ☐ Il s'exécute dans le Cloud sur les serveur de Microsoft
-  En effet, le code s'exécute dans le moteur du debugger directement : il faut donc faire attention, car les performances de l'ordinateur sont bien meilleures que celles du terminal mobile. Il faut donc penser à toujours bien tester son code dans le mode sans debugger pour vérifier son fonctionnement en conditions réelles.

Exercice

Comment place-t-on un `breakpoint` dans le code ?

- ☒ En cliquant dans le debugger à côté de la ligne sur laquelle on souhaite s'arrêter
- ☒ En ajoutant l'instruction `debugger` dans le code
-  Les deux méthodes sont possibles.

Exercice

Via le debugger, je peux avancer ligne par ligne dans le code.

- ☒ Vrai
- ☐ Faux
-  Le debugger donne la possibilité d'avancer pas à pas dans le code.

Exercice

Qu'est-ce que la *call stack* ?

- ⦿ C'est la liste qui présente l'ordre d'imbrication des appels de fonctions qui ont mené le programme au point où il en est
- C'est la liste des variables utilisées dans le scope courant
- 🔍 La *call stack* ou « pile d'appels » permet de tracer l'origine de l'appel d'une instruction de code.

Exercice

On peut naviguer dans la *call stack*.

- ⦿ Vrai
- Faux, elle n'est là qu'à titre informatif
- 🔍 On peut naviguer dans la *call stack*, même si l'on souhaite rejouer des instructions de code qui ont déjà eu lieu en amont. Cela permet de réévaluer un morceau de script pour mieux le debugger.

Exercice p. Solution n°4

```

1 import React, { useState } from "react";
2 import { View, Text, StyleSheet, Button } from "react-native";
3 import Constants from "expo-constants";
4 import { TextInput } from "react-native-gesture-handler";
5
6 export default function App() {
7   const [numberToFoundInput, setNumberToFoundInput] = useState();
8   const [numberToFound, setNumberToFound] = useState(null);
9   const [userAnswerInput, setUserAnswerInput] = useState();
10
11   // Première erreur corrigée
12   if (numberToFound === null) {
13     return (
14       <View style={styles.container}>
15         <TextInput
16           key="choose-number-to-found"
17           placeholder="Saisir le nombre mystère qu'il faudra trouver"
18           value={numberToFoundInput}
19           onChangeText={(val) => setNumberToFoundInput(val)}
20         />
21         <Button
22           title="Lancer le jeu"
23           onPress={() => {
24             setNumberToFound(Number(numberToFoundInput));
25           }}
26         />
27       </View>
28     );
29   }
30
31   const numberFoundByUser =
32     getMessageFromNumber(numberToFound, userAnswerInput) ===
33     "Bravo vous avez trouvé le nombre mystère !"; // Deuxième erreur corrigée
34
35   return (
36     <View style={styles.container}>
37       <TextInput
38         key="user-answer"

```

```

39     placeholder="Saisir le nombre pour essayer de trouver le nombre mystère"
40     value={userAnswerInput}
41     onChangeText={(val) => setUserAnswerInput(val)}
42   />
43   <Text style={styles.text}>
44     {getMessageFromNumber(numberToFound, userAnswerInput)}
45   </Text>
46   {numberFoundByUser && (
47     <Button
48       title="Réinitialiser"
49       onPress={() => {
50         setNumberToFound(null);
51         setUserAnswerInput(null);
52         setNumberToFoundInput(null);
53       }}
54     />
55   )}
56 </View>
57 );
58 }
59
60 function getMessageFromNumber(numberToFound, currentValue) {
61   // Troisième erreur corrigée
62   // On s'assure qu'on joue bien avec des nombres
63   numberToFound = Number(numberToFound);
64   currentValue = Number(currentValue);
65
66   if (typeof currentValue === "undefined" || currentValue === null) {
67     return "";
68   }
69
70   if (numberToFound === currentValue) {
71     return "Bravo vous avez trouvé le nombre mystère !";
72   }
73
74   // Quatrième erreur corrigée
75   return numberToFound < currentValue ? "C'est moins !" : "C'est plus";
76 }
77
78 const styles = StyleSheet.create({
79   container: {
80     flex: 1,
81     paddingTop: Constants.statusBarHeight,
82   },
83   text: {
84     textAlign: "center",
85     fontSize: 24,
86     marginVertical: 20,
87   },
88 });

```

Le première chose à remarquer est que l'écran permettant de saisir le nombre à trouver et le bouton pour lancer le jeu ne s'affichent pas : cela est dû au fait que la condition vérifiant que `numberToFound` est renseigné est fausse.

```
1 if (typeof numberToFound === "null") {
```

Cette instruction doit devenir :

```
1 if (numberToFound === null) {
```

Le bug suivant est une simple faute de frappe :

```
1 "Bravo vous avez trouvé le nombre mystèr"
```

Que l'on corrige par :

```
1 "Bravo vous avez trouvé le nombre mystère"
```

Le bug suivant intervient lorsque l'on saisit la bonne valeur du nombre à trouver. Il est attendu que l'on affiche le message « *Bravo vous avez trouvé le nombre mystère !* », mais c'est le message « *C'est moins !* » qui s'affiche.

C'est la conséquence du typage des variables utilisées pour la comparaison. Un correctif consiste donc à les convertir en nombres :

```
1 function getMessageFromNumber(numberToFound, currentValue) {
2   numberToFound = Number(numberToFound);
3   currentValue = Number(currentValue);
4   [...]
```

On peut aussi constater que les messages d'indices ne s'affichent pas correctement : lorsqu'on saisit une valeur plus petite, le message affiché est « *C'est moins !* » et, lorsqu'on saisit une valeur plus grande, le message est « *C'est plus* ». Cela devrait être l'inverse. L'erreur se situe au niveau du test entre les deux valeurs, qui est inversé par rapport à ce qu'il devrait être. Cela peut être corrigé par :

```
1 return numberToFound < currentValue ? "C'est moins !" : "C'est plus";
```